FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES

# CS054
# (INTELLIGENT SYSTEMS)

## EXERCISE

# 2

## Sentiment Analysis

| | |
|---|---|
| **Name:** | Tyrael Lance Peralta |
| **Section:** | AN42 |
| **Professor:** | Crisola Tan |

## I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design and evaluate solutions for computing problems. [PO: e]

## II. COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- To design an Intelligent System based on methods learned. [CLO: 3]

## III. INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Demonstrate understanding of Sentiment Analysis.
- Design and develop a simple program about Sentiment Analysis using Python.

## IV. BACKGROUND INFORMATION

Sentiment Analysis can assist us in determining the mood and feelings of the general public as well as obtaining useful information about the setting. Sentiment Analysis is the process of assessing data and categorizing it according to the research's needs.

Sentiments can be used to gain a better understanding of different events and their consequences. It goes by various names, including "sentiment analysis," "opinion mining," "opinion extraction," "sentiment mining," "subjectivity analysis," "affect analysis," "emotion analysis," and "review mining," but they all serve the same objective. We might be able to figure out what people like or want by evaluating their feelings.



**TextBlob**

TextBlob is a python library and offers a simple API to access its methods and perform basic NLP tasks. Textblob's sentiment function returns polarity and subjectivity as two characteristics.

The polarity ranges from [-1,1], with 1 denoting a positive statement and -1 denoting a negative one. Subjective statements usually refer to personal feelings, emotions, or judgments, whereas objective sentences refer to facts. Subjectivity is also a float with a value between 0 and 1.

## V. GRADING SYSTEM / RUBRIC (please see separate sheet)

## VI. LABORATORY ACTIVITY

**Perform the following:**

1. Add the following code:

```
from textblob import TextBlob
```

2. Declare a variable that will accept a sentence:

```
sentence = input("Enter a sentence : ")
```

3. Next initialize TextBlob

```
s = TextBlob(sentence)
```

4. Then check the polarity of the sentence by using the code below:

```
p = s.sentiment.polarity
```

5. Make a decision based on the polarity. If the polarity of the sentence is less than zero the sentiment is negative. If the polarity is equals to zero, then the sentiment is neutral. If the polarity is greater than zero, the sentiment is positive.

```
#Negative p<0, Neutral p=0, Positive p>0
if p <0:
    print("Negative")
elif p==0:
    print("Neutral")
elif p>0 and p<=1:
    print("Positive")
```

6. Make the program perform the sentiment analysis 10 times and make a summary of how many sentences are positive, neutral or negative.

**OUTPUT:**

```
Do you want to analyze a sentence? (yes/no): yes
Enter a sentence: I like it
The sentiment of the sentence is neutral.

Do you want to analyze a sentence? (yes/no): yes
Enter a sentence: I love it when it rains
The sentiment of the sentence is positive.

Do you want to analyze a sentence? (yes/no): yes
Enter a sentence: You need to step up your game!
The sentiment of the sentence is negative.

Do you want to analyze a sentence? (yes/no): yes
Enter a sentence: I've seen you sad and I don't want that
The sentiment of the sentence is negative.

Do you want to analyze a sentence? (yes/no): yes
Enter a sentence: Oh... well that just happened...
The sentiment of the sentence is neutral.

Do you want to analyze a sentence? (yes/no): yes
Enter a sentence: I upgraded my pc!
The sentiment of the sentence is neutral.

Do you want to analyze a sentence? (yes/no): yes
Enter a sentence: I don't like playing your games, answer me!
The sentiment of the sentence is neutral.

Do you want to analyze a sentence? (yes/no): yes
Enter a sentence: I hate coffee, it tastes like mud!
The sentiment of the sentence is negative.

Do you want to analyze a sentence? (yes/no): yes
Enter a sentence: I think boss runs is good game design, you just salty.
The sentiment of the sentence is positive.

Do you want to analyze a sentence? (yes/no): yes
Enter a sentence: I hate when you look at others!
The sentiment of the sentence is negative.
```

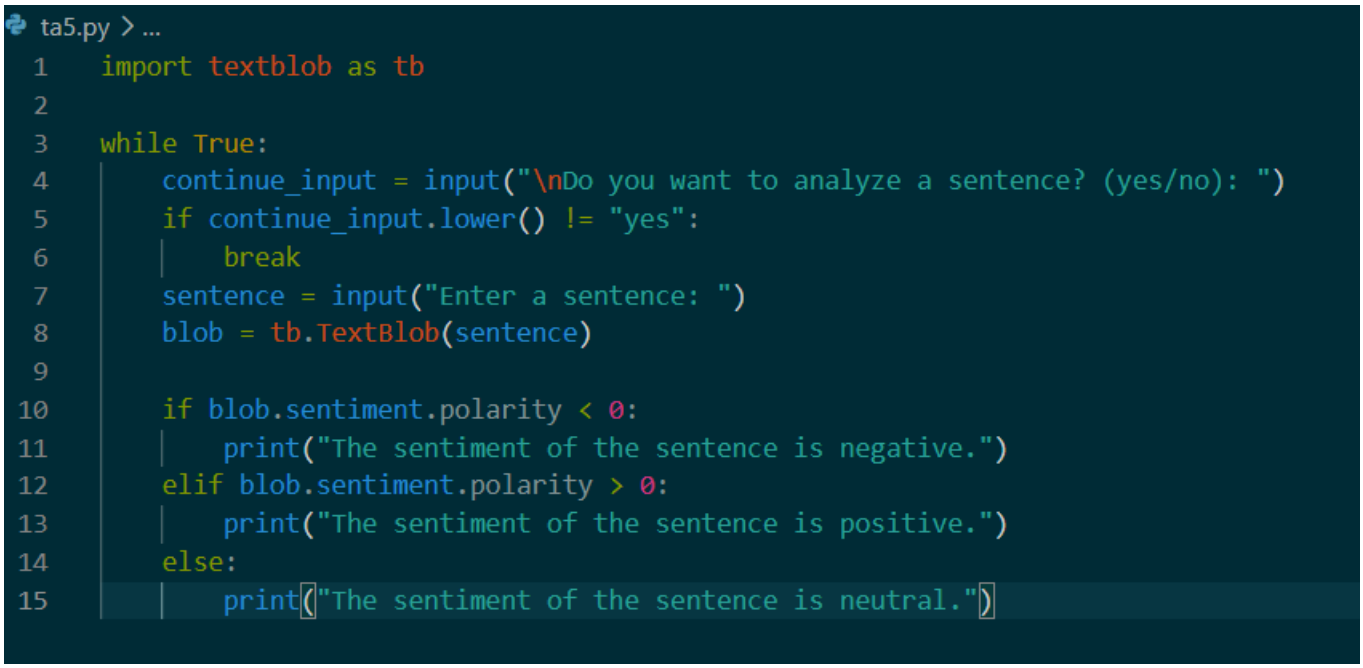
**CODE:**

```
import textblob as tb

while True:
    continue_input = input("\nDo you want to analyze a sentence? (yes/no): ")
    if continue_input.lower() != "yes":
        break
    sentence = input("Enter a sentence: ")
    blob = tb.TextBlob(sentence)

    if blob.sentiment.polarity < 0:
        print("The sentiment of the sentence is negative.")
    elif blob.sentiment.polarity > 0:
        print("The sentiment of the sentence is positive.")
    else:
        print("The sentiment of the sentence is neutral.")
```

**CODE SCREENSHOT:**




## VII. CHALLENGE

Write a program that will ask the user for a string containing a sentence. The program will then compute the happiness and sadness level of the sentence using the two functions described below. If the happiness level is higher than sadness level, then the tone of the sentence is happy. If the sadness level is higher, then the tone of the sentence is sad. Otherwise, it is neutral.

Find and print the tone of the sentence by printing the number of happy words followed by the number of sad

words, followed by a simple statement of the analysis (whether positive, negative or neutral).

To accomplish this, you will write a function called number_happy(sentence) which returns the number of words in a given string called “sentence” that are happy. To do this, look for a public dataset containing a list of happy words. Find the total number of words in “sentence” that are in the happy words dataset.

Next, write a second function called number_sad(sentence) that works the same way but instead counts the number of sad words, also from a separate dataset.

## OUTPUT:

```
lSystems/TechAssesments/TA1/ta5.py'
Happy Count: 80 | Words: ['unique', 'many', 'largely', 'latest', 'far', 'largely', 'old', 'apparent', 'many', 'much', 'very', 'most', 'much', 'huge', 'large', 'quickly', 'most', '
interesting', 'largely', 'able', 'exactly', 'main', 'able', 'fit', 'fly', 'special', 'new', 'new', 'far', 'fun', 'generally', 'best', 'more', 'powerful', 'large', 'kind', 'funny',
 'first', 'able', 'old', 'quickly', 'generally', 'really', 'more', 'most', 'cool', 'really', 'whole', 'fun', 'major', 'whole', 'directly', 'freely', 'exactly', 'many', 'new', 'dir
ectly', 'right', 'more', 'fine', 'near', 'generally', 'best', 'action', 'full', 'really', 'very', 'early', 'able', 'quickly', 'generally', 'most', 'quickly', 'complete', 'fun', 'o
ld', 'much', 'real', 'better', 'free']
Sad Count: 51 | Words: ['game', 'extremely', 'uninspired', 'poorly', 'few', 'extremely', 'flat', 'dead', 'flat', 'game', 'game', 'due', 'other', 'game', 'extremely', 'tedious', 'd
ead', 'few', 'game', 'little', 'other', 'game', 'game', 'least', 'devastating', 'game', 'behind', 'unable', 'behind', 'extremely', 'annoying', 'behind', 'poor', 'extremely', 'curi
ous', 'commonly', 'behind', 'few', 'killed', 'extremely', 'annoying', 'slow', 'down', 'bored', 'boring', 'annoying', 'frustrating', 'game', 'forced', 'annoying', 'green']
The sentiment of the sentence is positive.
(.venv) PS C:\Files\School\Fourth\IntelSystems\TechAssesments\TA1>
```

## CODE:

```
import textblob as tb

def number_happy(sentence):
    blob = tb.TextBlob(sentence)
    positive_words = []
    happy_count = 0

    for word in blob.words:
        word_blob = tb.TextBlob(word.lower())
        if word_blob.sentiment.polarity > 0:
            happy_count += 1
            positive_words.append(word.lower())

    return happy_count, positive_words

def number_sad(sentence):
    blob = tb.TextBlob(sentence)
    negative_words = []
    sad_count = 0

    for word in blob.words:
        word_blob = tb.TextBlob(word.lower())
        if word_blob.sentiment.polarity < 0:
            sad_count += 1
            negative_words.append(word.lower())

    return sad_count, negative_words

#Source: IGN's GOD HAND review
#https://www.ign.com/articles/2006/10/10/god-hand-review

sentence = """Clover Studio has a knack for creating unique and off-the-wall games, many of which have ties to 2D art and gameplay.
Viewtiful Joe was largely a side-scroller set in a 3D world with a cartoony art style and time-altering mechanics. The equally-excellent
Okami utilized painting techniques not only for its visuals but core gameplay mechanics as well. And now we're presented with God Hand,
the studios' latest release and also its oddest so far, largely mimicking the brawlers from the old days of gaming in a kooky 3D world.
It's somewhat apparent that God Hand was meant to be a joke on many levels, much as a film director might intentionally make a B-movie.
The game practically encourages button mashing, enemies are extremely generic, the level layout is very uninspired, the dialog and jokes are
poorly written and delivered, and most of all, much of the control and gameplay mechanics are so old-school it hurts. While Clover took a
huge risk in hoping that this sort of off-kilter design would appeal to some gamers, and there will probably be a few folks out there who will
dig this extremely niche design, by and large the joke simply falls flat. Dead flat.

Put it this way: Had this game not been designed by Clover Studio, released by Capcom and hyped (to some degree) because of this, it would
be quickly passed over and shunned by most who play it.

One of the only interesting aspects of the game is its combat system, largely in part due to the fact that you're able to customize exactly
which moves you want in your combo string. While you only have one main combo chain, you're able to fit in other moves that you've
```

**assigned to the face buttons and create attack chains on the fly. Some moves have special attributes, like juggling an opponent, tossing them away or breaking a block. This customizable aspect, and the fact that you'll earn and buy new moves as the game progresses, means that you'll constantly be switching up your attack style and incorporating new moves.**

**But that's about as far as the fun of combat goes. Beyond this, it's an extremely tedious brawler where you'll generally mash a button or two until an enemy is dead. There are a few tactics of sorts mixed in, like choosing the best times to use a slower but more powerful move, but in large part you'll be mashing buttons until everyone is finished. It's kind of funny the first time you're able to stomp on someone's head with your leg moving at mach 3, but this gets old quickly.**

**The God Hand aspect of the game refers to Gene's arm, which you can unleash when powered up and generally kick the hell out of everything for a short while. This really is little more than a power-up attack that you'll find in most any other combat-heavy game around and doesn't save the game in the least since all it does is strengthen your attacks for a handful of seconds. The Roulette Wheel is somewhat similar on a couple levels in that these moves allow you to perform some sort of devastating attack, but they're one-off moves that while looking cool don't really add a whole lot of fun to the combat system. All of these are simply eye candy and the game's reliance on these mechanics simply doesn't hold up.**

**One of the major problems with the combat, and God Hand on the whole in fact, is that the camera is stuck directly behind your character and you're unable to freely control it. Where you walk is exactly where it points, so enemies will often come up (or even appear) behind you, and this can be extremely annoying in a crowded room. On many occasions we finished off a handful of enemies only to have new foes appear directly behind us and take us out. The right analog stick acts as a dodge button, so since you don't have a block of any sort you'll be using this often, but because of the poor camera you'll often perform backflips into nearby walls.**

**There's an extremely curious health system at work as well. To regain life, you need to pick up fruits from inside boxes, or more commonly, left behind by downed enemies. This part is fine, though your health doesn't refill between levels. This means that if you were near death at the end of the last stage, your only hope is to kill a few guys at the beginning of the next stage and hope they drop some grub. Generally the best course of action here is to simply let yourself be killed and choose to continue with a full life bar. It really doesn't make any sense that your health isn't refilled since you're very likely to die a minute later anyway and then restart with no penalty. And of course, you have to wait through a load screen when you continue.**

**Enemy design is extremely generic with demons and such that reek of early '90s arcade brawlers. While some of them have varying combat skills, like being able to quickly hop up when thrown to the ground or an affinity for dodging, you generally just run at them, punch away until they're about to attack and then flip away in cowardice. Rinse and repeat.**

**The most annoying aspect about this is that you'll often have to fight demon spirits that arise from fallen enemies. They take a while to kill and simply act as roadblocks to slow down your progression. After fighting them for the 3rd time you'll become bored, and by the 50th you'll be throwing your controller across the room.**
**Verdict**
**The bottom line is that God Hand quickly becomes a boring, annoying and frustrating game. Why should you be forced to try and complete a level with no starting health when you have no hope of survival and will just need to reload a minute later? What's fun in fighting the same annoying demon creatures over and over again when the fight was old the second time you tried it? Why, oh why, did Clover Studio put so much money into such a risky joke, and why did no one see that the joke didn't have a real punch line at any point in the development? Here's a better joke for free: What's green and has wheels?**
**Grass. I lied about the wheels.**

**"""**

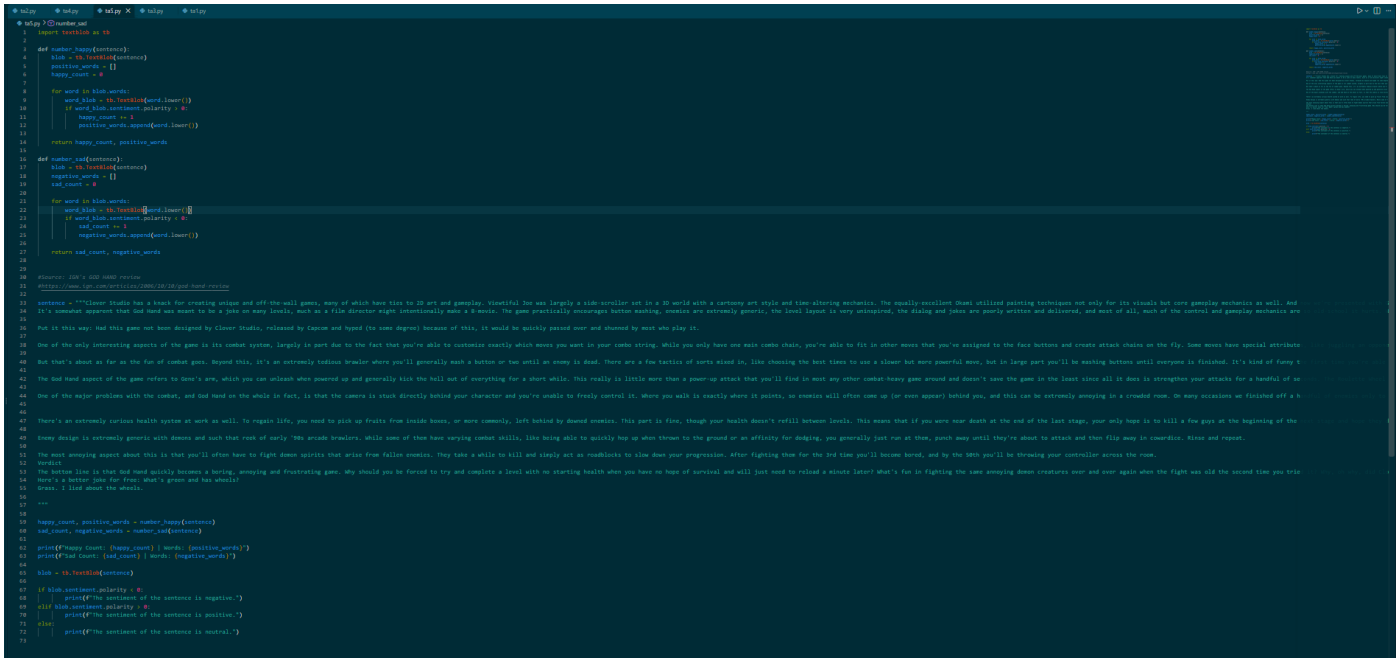
```
happy_count, positive_words = number_happy(sentence)
sad_count, negative_words = number_sad(sentence)

print(f"Happy Count: {happy_count} | Words: {positive_words}")
print(f"Sad Count: {sad_count} | Words: {negative_words}")

blob = tb.TextBlob(sentence)

if blob.sentiment.polarity < 0:
    print(f"The sentiment of the sentence is negative.")
elif blob.sentiment.polarity > 0:
    print(f"The sentiment of the sentence is positive.")
else:
    print(f"The sentiment of the sentence is neutral.")
```

## CODE SCREENSHO



## VIII. REFERENCES


- Artasanchez A. Joshi P.(2020). Artificial Intelligence with Python: You Complete Guide to Building Intelligent apps using Python 3.x 2nd Edition. Packt Publishing
- Attaway, Stormy (2013). MATLAB ®. A Practical Introduction to Programming and Problem Solving Third Edition. Elsevier, Inc.

Online Sources:

- https://www.cuelogic.com/blog/role-of-python-in-artificial-intelligencehttps://monkeylearn.com/sentiment-analysis
- https://towardsdatascience.com/a-beginners-guide-to-sentiment-analysis-in-python-95e354ea84f6
- https://research.google.com/colaboratory
- https://www.youtube.com/watch?v=RLYoEyIHL6A